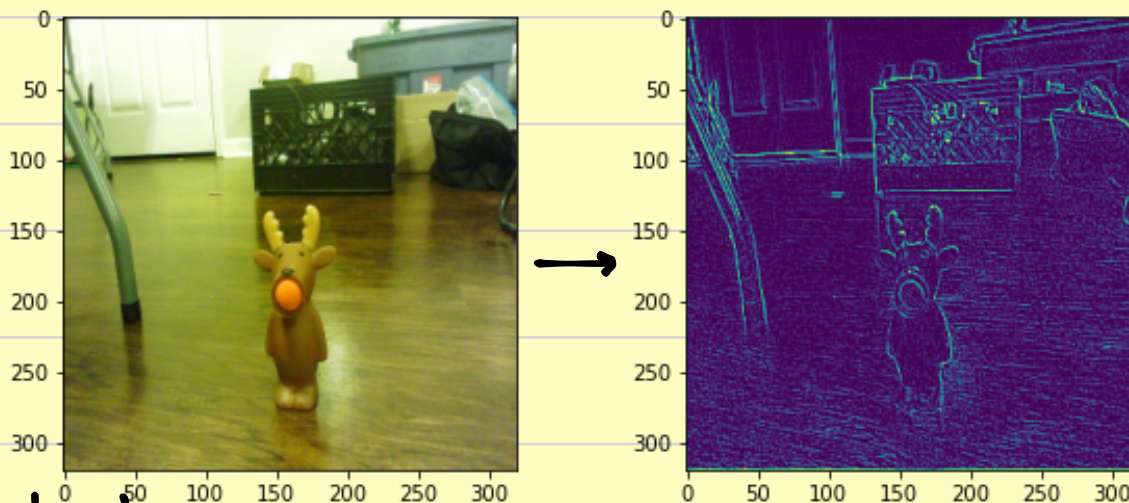


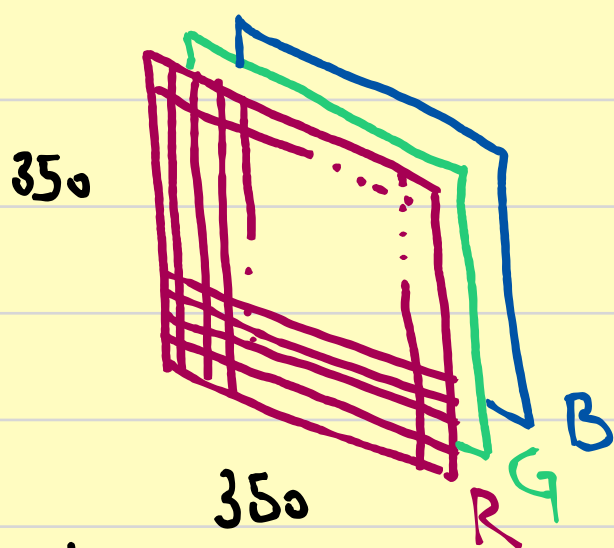
Deep Learning by Hand (2)

• CONVOLUTION •



- Convolution ist eine Technik, angewendet in der Verarbeitung von Euklydischen Datensätzen (Bilder, Videos, ...) um Verhaltensmuster aus den Daten herauszufiltern.
- Die Convolution wendet ..FILTERN" an, um relevante Informationen aus den Daten zu ziehen.
- Diese ..FILTER" nennen wir ..KERNELS".
In diesem Kontext ein kernel ist ein ..TENSOR" welche die Gewichte der Convolution beeinflusst.

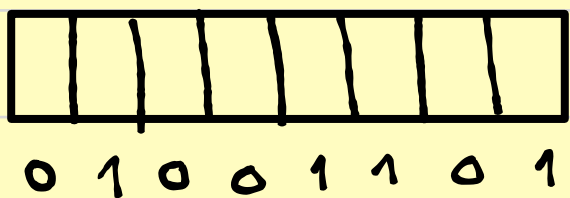
WAS IST EIN FARBBILD?



Jeder Pixel hat 3 Werte, einmal Kanal Rot, einmal Grün, einmal Blau.

$P_{i,j} [R, G, B]$

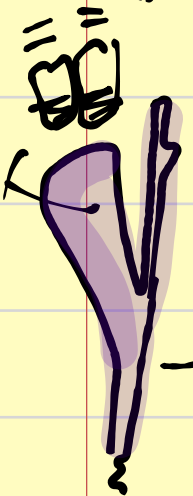
Die Werte können zw. $[0, 255]$ sein.



8 bits $\equiv$ 1 byte $[0, 1]$ $2^8 \equiv 256$

• Das Beispiel oben hat 350 pixels in der x axis, 350 in der y und ist somit ein „Tensor“ mit Dimensionen $[350, 350, 3]$

• In einem Euklydischen Datensatz können wir einen sinnvollen Abstand messen: in dem Beispiel oben, alle orangene Pixels der Obst sind nah beieinander und so können wir die Orange erkennen.



• Convolution kann nur in einem Euklydischen Domäne angewendet werden.

Input Filter / kernel . Eigenschaften .

- 1) können unterschiedliche Formen haben .
- 2) Die kernel machen filtern aus, diese sind Parameter der Convolutional layers (bei Deep learning)
- 3) Es gibt kernel für :
 - Aufmerksamkeit
 - Blurring

Konturerkennung

...

Beispiele: 1) Aufmerksamkeitskernel (1) $[3 \times 3]$

0	-1	0
-1	4	-1
0	-1	0

2) Stärkere Aufmerksamkeit (2) $[3 \times 3]$

0	-1	0
-1	8	-1
0	-1	0

3) Kantenerkennung $[3 \times 3]$

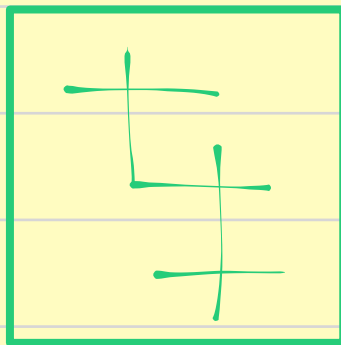
1	0	-1
0	0	0
-1	0	1

Anwendung der Konvolution

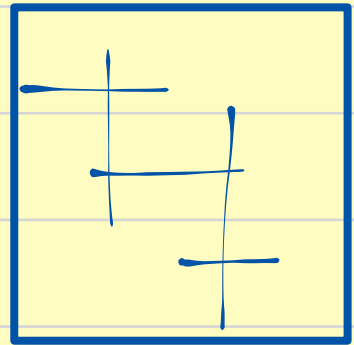
R

1	1	0	2
1	2	1	1
0	1	2	1
1	0	0	1

G



B



STRIDE: wie viel sich der kernel bewegt

STRIDE = 1 pixel

Aufmerksamkeitskernel:

0	-1	0
-1	4	-1
0	-1	0

R

K

$$\begin{array}{cccc}
 1 & 1 & 0 & 2 \\
 1 & 2 & 1 & 1 \\
 0 & 1 & 2 & 1 \\
 1 & 0 & 0 & 1
 \end{array}$$

$$\begin{array}{ccc}
 0 & -1 & 0 \\
 -1 & 4 & -1 \\
 0 & -1 & 0
 \end{array}$$

$$\begin{aligned}
 &1 \cdot 0 + 1 \cdot (-1) + 0 \cdot 0 + \\
 &1 \cdot (-1) + 2 \cdot 4 + 1 \cdot (-1) + \\
 &0 \cdot 0 + 1 \cdot (-1) + 2 \cdot 0 = 4
 \end{aligned}$$

S=1

$$\begin{array}{cccc}
 1 & 1 & 0 & 2 \\
 1 & 2 & 1 & 1 \\
 0 & 1 & 2 & 1 \\
 1 & 0 & 0 & 1
 \end{array}$$

$$\begin{array}{ccc}
 0 & -1 & 0 \\
 -1 & 4 & -1 \\
 0 & -1 & 0
 \end{array}$$

$$\begin{aligned}
 &1 \cdot 0 + 0 \cdot (-1) + 2 \cdot 0 + \\
 &2 \cdot (-1) + 4 \cdot 1 + 1 \cdot (-1) + \\
 &1 \cdot 0 + 2 \cdot (-1) + 1 \cdot 0 = -1
 \end{aligned}$$

4	-1
0	5

$$\begin{array}{cccc}
 1 & 1 & 0 & 2 \\
 1 & 2 & 1 & 1 \\
 0 & 1 & 2 & 1 \\
 1 & 0 & 0 & 1
 \end{array}$$

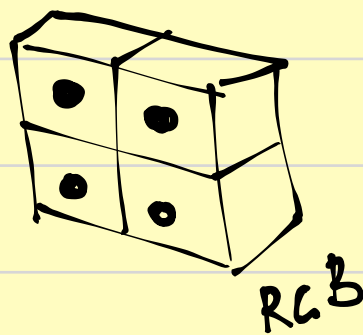
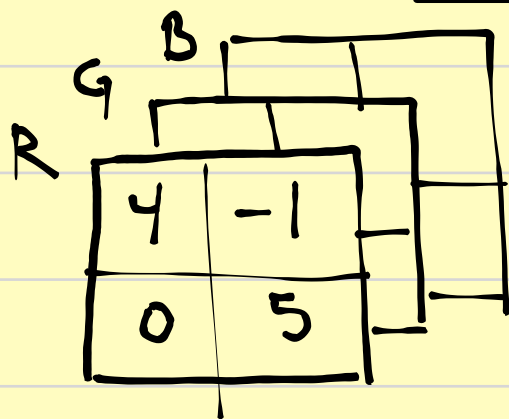
$$\begin{array}{ccc}
 0 & -1 & 0 \\
 -1 & 4 & -1 \\
 0 & -1 & 0
 \end{array}$$

$$\begin{aligned}
 &1 \cdot 0 + 2 \cdot (-1) + 1 \cdot 0 + \\
 &0 \cdot (-1) + 4 \cdot 1 + 2 \cdot (-1) + \\
 &1 \cdot 0 + (-1) \cdot 0 + 0 \cdot 0 = 0
 \end{aligned}$$

$$\begin{array}{cccc}
 1 & 1 & 0 & 2 \\
 1 & 2 & 1 & 1 \\
 0 & 1 & 2 & 1 \\
 1 & 0 & 0 & 1
 \end{array}$$

$$\begin{array}{ccc}
 0 & -1 & 0 \\
 -1 & 4 & -1 \\
 0 & -1 & 0
 \end{array}$$

$$\begin{aligned}
 &2 \cdot 0 + 1 \cdot (-1) + 1 \cdot 0 + \\
 &1 \cdot (-1) + 2 \cdot 4 + (-1) \cdot 1 + \\
 &0 \cdot 0 + 0 \cdot (-1) + 0 \cdot 1 = 5
 \end{aligned}$$



4x4
kanal

4 Rechnungen

3x3 + s=1
kernel

2x2

Convolution

Konvolutionskernel 2×2

2	0
-1	1

$s=1$

1 1 0 2

1 2 1 1

0 1 2 1

1 0 0 1

2 0

-1 1

3	1	0
3	5	1
-1	2	5

4×4

Kanal

9 Rechnungen

3×3

Convolution

2×2 + $s=1$
kernel

Mehr Auflösung mit mehr Rechenleistung.

Konvolutionskernel 2×2

2	0
-1	1

$s=2$

1 1 0 2

1 2 1 1

0 1 2 1

1 0 0 1

2 0

-1 1

3 0

-1 5

4×4

Kanal

4 Rechnungen

2×2

Convolution

2×2 + $s=2$
kernel

PADDING

R

0	0	0	0	0	0
0	1	1	0	2	0
0	1	2	1	1	0
0	0	1	2	1	0
0	1	0	0	1	0
0	0	0	0	0	0

s=2

kernel

0	-1	0
-1	4	-1
0	-1	0

p=1 : addieren von p Umrandung im Wert von 0
(NULL)

2x2 Convolution

2	-4
-3	5

